

实验 4 进程通信 II

4.1 实验目的

- (1) 了解共享存储区通信的系统调用 `shmget()`、`shmat()`、`shmdt()`、`shmctl()`的功能；
- (2) 了解消息队列通信的系统调用 `msgget()`、`msgsnd()`、`msgrcv()`、`msgctl()`的功能；
- (3) 理解进程之间通信的基本原理；
- (4) 掌握利用共享存储区和消息缓冲队列进行进程间通信的实现方法。

4.2 预备知识

在 Linux 系统中，进程与进程之间需要互相交换数据，即进程通信。进程通信可以发生在同一主机的进程之间，也可以发生在不同主机与主机的进程之间。

Linux 下进程间通信的几种主要手段有：(1)无名管道(Pipe)及有名管道(named pipe)、(2)信号(Signal)、(3)队列(消息队列)、(4)共享内存、(5)信号量和(6)套接口(Socket)。

共享存储区 (Share Memory) 通信机制可使若干进程共享主存中的某一个区域，且使该区域出现 (映射) 在多个进程的虚地址空间中。另一方面，一个进程的虚地址空间中又可连接多个共享存储区，每个共享存储区都有自己的名字。当进程间欲利用共享存储区进行通信时，必须先在主存中建立共享存储区，然后将它附接到自己的虚地址空间上。此后，进程对该区的访问操作，与对其虚地址空间的其它部分的操作完全相同。进程之间便可通过对共享存储区中数据的读、写来进行直接通信。

共享存储区机制只为进程提供了用于实现通信的共享存储区和对共享存储区进行操作的手段，然而并未提供对该区进行互斥访问及进程同步的措施。因而当用户需要使用该机制时，必须自己设置同步和互斥措施才能保证实现正确的通信。

消息队列是消息的链接表，可以把消息看作一个记录，具有特定的格式以及特定的优先级。有写权限的进程可以向消息队列中添加新消息；有读权限的进程则可以从消息队列中读走消息。消息本质上是一种数据结构，存放在内核中，并由消息队列标识符标识。`msgget` 用于创建一个新队列或打开一个现存的队列。`msgsnd` 用于将新消息添加到队列尾端。每个消息包含一个正长整型类型字段，一个非负长度以及实际数据字节 (对应于长度)，所有这些都在将消息添加到队列时，传送给 `msgsnd`。`msgrcv` 用于从队列中取消息。

4.3 实验内容

1.编写一段程序，使其用共享存储区来实现父子进程之间的进程通信。父进程创建一个长度为 512 字节的共享内存空间，显示写入该共享内存的数据；子进程将共享内存也附加到自己的地址空间，并向共享内存中写入数据。

2.编写一段程序，使其用消息缓冲队列来实现父进程和子进程之间的通信。父进程先建立一个关键字为 `MSGKEY`(如 75) (即 `#define MSGKEY 75`) 的消息队列，然后等待接收类型为 1 的消息；在收到请求消息后，它便显示字符串 “serving for client ” 和接收到的子进程的进程标识数，表示正在为子进程服务；然后再向子进程发送一条应答消息，该消息

的类型是该子进程的进程标识数，而正文则是父进程自己的标识数。子进程则向消息队列发送类型为 1 的消息(消息的正文为自己的进程标识数)，以取得父进程的服务，并等待父进程发来的应答；然后显示字符串“receive reply from ”和接收到的父进程的标识数。

4.4 实验指导

1.共享存储区通信涉及的 API

(1) int shmget(key, size, flag)

得到一个共享内存标识符或创建一个共享内存对象并返回共享内存标识符。其中，key 为 0(IPC_PRIVATE)时，表示创建新共享内存对象；key 为大于 0 的 32 位整数时，视参数 flag 确定操作。

size 是大于 0 的整数时，表示新建的共享内存大小，以字节为单位；size 取值为 0 时，表示获取共享内存对象。

flag 是用户设置的标志，为 0 时表示获取共享内存标识符；为 IPC_CREAT 时，表示若系统中尚无指定的共享存储区，则由核心建立一个共享存储区；若系统中已有共享存储区，便忽略 IPC_CREAT。为 IPC_CREAT|IPC_EXCL 时，如果内存中不存在键值与 KEY 相等的共享内存，则新建一个共享内存；如果不存在，则报错。

(2) char *shmat(shmid, addr, flag)

共享存储区的附接。从逻辑上将一个共享存储区附接到进程的虚拟地址空间上。该系统调用的返回值是共享存储区所附接到的进程虚地址 viraddr。

其中，shmid 是共享存储区的标识符。

addr 是用户给定的，将共享存储区附接到进程的虚地址空间，指定共享内存出现在进程内存地址的什么位置，直接指定为 NULL 让内核自己决定一个合适的地址位置。

flag 规定共享存储区的读、写权限，以及系统是否应对用户规定的地址做舍入操作。其值为 SHM_RDONLY 时，表示只能读；其值为 0 时，表示可读、可写。

(3) int shmdt(viraddr)

把一个共享存储区从指定进程的虚地址空间断开。其中，viraddr 是要断开连接的虚地址，亦即以前由连接的系统调用 shmat() 所返回的虚地址。调用成功时，返回 0 值，调用不成功，返回-1。值得注意的是：这里的 addr 我们可以用于存数据。

(4) int shmctl(shmid, cmd, buf)

共享存储区的控制，对其状态信息进行读取和修改。其中，buf 是用户缓冲区地址，cmd 是操作命令。命令可分为多种类型：

- ①查询共享存储区情况。如其长度、当前连接的进程数、共享区的创建者标识符等；
- ②设置或改变共享存储区的属性。如共享存储区的许可权、当前连接的进程计数等；
- ③对共享存储区的加锁和解锁命令；
- ④删除共享存储区标识符等。

上述的查询是将 shmid 所指示的数据结构中的有关成员，放入所指示的缓冲区中；而设置是用由 buf 所指示的缓冲区内容来设置由 shmid 所指示的数据结构中的相应成员。

cmd 有下列几种数值：

IPC_STAT 得到共享内存的状态。把共享内存的 shmid_ds 结构复制到 buf 中。

IPC_SET 改变共享内存的状态，将参数 buf 所指的 shmid_ds 结构中的 uid、gid 和 mode 复制到共享内存的 shmid_ds 结构内。

IPC_RMID 删除共享内存和数据结构。

SHM_LOCK 不让此共享内存置换到 swap；SHM_UNLOCK 允许此共享内存置换到

swap。SHM_LOCK 和 SHM_UNLOCK 为 LINUX 特有，且唯有超级用户（root）允许使用，而我们常用的是删除共享内存：shmctl(shmid, IPC_RMID, NULL)。

2. 消息队列通信所涉及的 API

(1)msgget 函数

该函数用来创建和访问一个消息队列。它的原型为：

```
int msgget(key_t key, int msgflg);
```

与其他的 IPC 机制一样，程序必须提供一个键来命名某个特定的消息队列。msgflg 是一个权限标志，表示消息队列的访问权限，它与文件的访问权限一样。msgflg 可以与 IPC_CREAT 做或操作，表示当 key 所命名的消息队列不存在时创建一个消息队列，如果 key 所命名的消息队列存在时，IPC_CREAT 标志会被忽略，而只返回一个标识符。

该函数返回一个以 key 命名的消息队列的标识符（非零整数），失败时返回-1。

(2)msgsnd 函数

该函数用来把消息添加到消息队列中。它的原型为：

```
int msgsnd(int msgid, const void *msg_ptr, size_t msg_sz, int msgflg);
```

msgid 是由 msgget 函数返回的消息队列标识符。

msg_ptr 是一个指向准备发送消息的指针，但是消息的数据结构却有一定的要求，指针 msg_ptr 所指向的消息结构一定要是以一个长整型成员变量开始的结构体，接收函数将用这个成员来确定消息的类型。所以消息结构要定义成这样：

```
1. struct my_message{
2.     long int message_type;
3.     /* The data you wish to transfer*/
4. };
```

msg_sz 是 msg_ptr 指向的消息的长度，注意是消息的长度，而不是整个结构体的长度，也就是说 msg_sz 是不包括长整型消息类型成员变量的长度。

msgflg 用于控制当前消息队列满或队列消息达到系统范围的限制时将发生的事情，取值为 IPC_NOWAIT 表示非阻塞发送方式；取值为 0 时表示阻塞发送方式。

若调用成功，消息数据的一份副本将被放到消息队列中，并返回 0，失败时返回-1。

(3)msgrcv 函数

该函数用来从一个消息队列获取消息，它的原型为

```
int msgrcv(int msgid, void *msg_ptr, size_t msg_sz, long int msgtype, int msgflg);
```

msgid, msg_ptr, msg_sz 的作用也函数 msgsnd 函数的一样。

msgtype 可以实现一种简单的接收优先级。如果 msgtype 为 0，就获取队列中的第一个消息。如果它的值大于零，将获取具有相同消息类型的第一个信息。如果它小于零，就获取类型等于或小于 msgtype 的绝对值的第一个消息。

msgflg 用于控制当队列中没有相应类型的消息可以接收时将发生的事情。取值为 IPC_NOWAIT 表示非阻塞接收方式；取值为 0 时表示阻塞接收方式。

调用成功时，该函数返回放到接收缓存区中的字节数，消息被复制到由 msg_ptr 指向的用户分配的缓存区中，然后删除消息队列中的对应消息。失败时返回-1。

(4)msgctl 函数

该函数用来控制消息队列，它与共享内存的 shmctl 函数相似，它的原型为：

```
int msgctl(int msgid, int command, struct msgid_ds *buf);
```

command 是将要采取的动作，它可以取 3 个值，

IPC_STAT：把 msgid_ds 结构中的数据设置为消息队列的当前关联值，即用消息队列的当前关联值覆盖 msgid_ds 的值。

IPC_SET: 如果进程有足够的权限, 就把消息队列的当前关联值设置为 msgid_ds 结构中给出的值。

IPC_RMID: 删除消息队列。

buf 是指向 msgid_ds 结构的指针, 它指向消息队列模式和访问权限的结构。

成功时返回 0, 失败时返回-1。

4.5 参考代码

1.共享存储区通信

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<sys/ipc.h>
#include<sys/shm.h>
#include<sys/wait.h>
#include<error.h>
#define SHMKEY 75
#define SIZE 512

int main()
{
    int shmid;
    char *shmaddr_child,*shmaddr_parent;
    char message[512];
    struct shm_id buf;
    int flag=0;
    int pid;

    shmid=shmget(IPC_PRIVATE,SIZE,IPC_CREAT|0600);

    if(shmid<0)
    {
        perror("get shm ipc_id error\n");
        return -1;
    }
}
```

```

pid=fork();
if(pid==0)
{
    shmaddr_child=(char *)shmat(shmid,NULL,0);
    if(shmaddr_child==(char *)-1)
    {
        perror("shmat addr error\n");
        return -1;
    }
    memset(message,0,512);
    sprintf(message,"%d is sending message to parent",getpid());
    puts(message);
    strcpy(shmaddr_child,message);
    shmdt(shmaddr_child);
    return 0;
}
else if(pid>0)
{
    wait(0);
    sleep(3);
    shmaddr_parent=(char *)shmat(shmid,NULL,0);
    if(shmaddr_parent==(char *)-1)
    {
        perror("shmat addr error\n");
        return -1;
    }
    printf("parent display: %s\n",shmaddr_parent);
    flag=shmctl(shmid,IPC_STAT,&buf);
    if(flag==-1)
    {
        perror("shmctl shm error\n");
        return -1;
    }
    printf("shm_segsz=%ld bytes\n",buf.shm_segsz);
    printf("parent pid=%d shm_cpid=%d\n",getpid(),buf.shm_cpid);
    printf("child pid=%d shm_lpid=%d\n",pid,buf.shm_lpid);
    shmdt(shmaddr_parent);
    shmctl(shmid,IPC_RMID,NULL);
}
else
{
    perror("fork error\n");
    shmctl(shmid,IPC_RMID,NULL);
}
return 0;
}

```

2.消息队列通信

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<sys/msg.h>
#include<sys/wait.h>

#define MSGKEY 75

typedef struct msgbuf{
    long mtype;
    char mtext[1024];
}msgbuf;

int main()
{
    int pid,c,s;
    int msqid;

    msgbuf sendermsgpc,recevermsgpc,sendermsgps,recevermsgps;

    memset(&sendermsgpc,0,1024);
    memset(&recevermsgpc,0,1024);
    memset(&sendermsgps,0,1024);
    memset(&recevermsgps,0,1024);

    msqid=msgget((key_t)MSGKEY,IPC_CREAT|0666);//create one message quence
    if(msqid<0)
    {
        printf("wrong in function of msgget\n");
        return 0;
    }
}
```

```

if((pid=fork())<0)
{
    printf("There have something wrong in the function of fork");
    return 0;
}
else if(pid==0)//child process
{
    c=getpid();
    printf("This is child process:%d\n",c);
    sendermsgpc.mtype=1;
    sprintf(sendermsgpc.mtext,"%d",c);
    msgsnd(msqid,&sendermsgpc,1024,0);

    sleep(3);

    msgrcv(msqid,&receivermsgpc,1024,0,0);
    printf("receive reply from server %s\n",receivermsgpc.mtext);
    printf("end of child process\n");
}
else//parent process
{
    s=getpid();
    printf("This is parent process:%d\n",s);

    sleep(3);

    msgrcv(msqid,&receivermsgps,1024,0,0);
    printf("serving for client %s\n",receivermsgps.mtext);

    sendermsgps.mtype=1;
    sprintf(sendermsgps.mtext,"%d",s);
    msgsnd(msqid,&sendermsgps,1024,0);
    printf("end of parent process\n");

    wait(0);
    msgctl(msqid,IPC_RMID,0);
}
return 0;
}

```